

What is Hashcat?

Hashcat is a popular and powerful password recovery tool that can help you recover lost passwords, crack hashes, and even perform advanced attacks against password-protected systems. It is known for its speed and versatility, making it a go-to tool for security professionals, researchers, and enthusiasts.

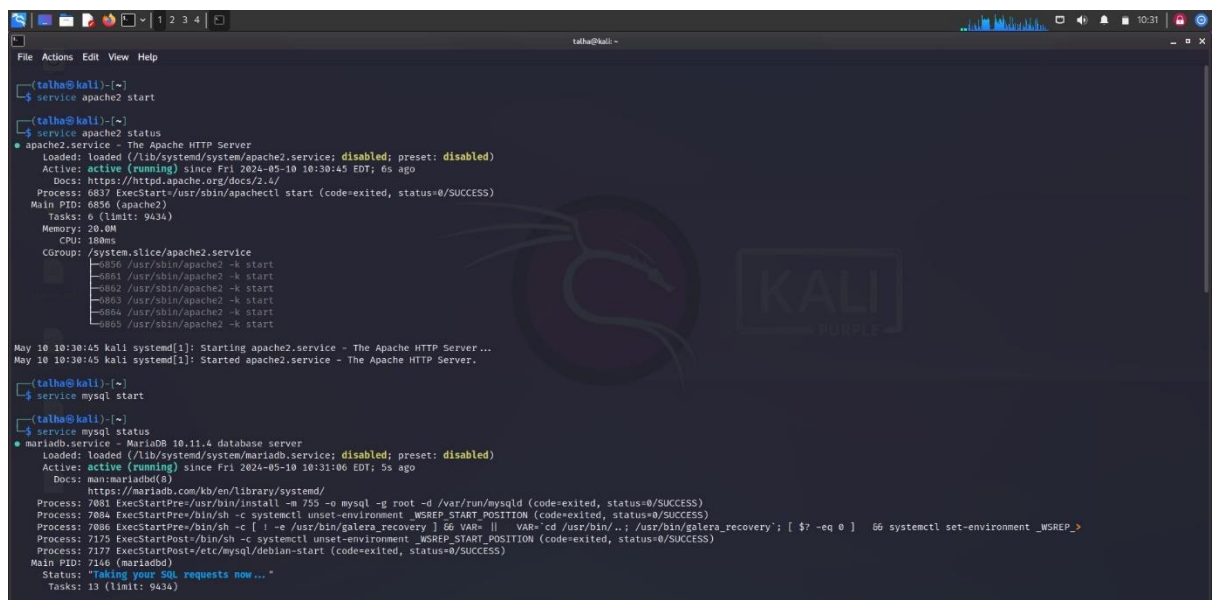
Why Use Hashcat?

1. **Password Recovery:** Hashcat can be used to recover lost passwords from a variety of hash types, including MD5, SHA-1, SHA-256, and many others.
2. **Password Cracking:** It can crack passwords by brute-forcing, using wordlists, and applying rules to generate possible passwords.
3. **Security Audits:** Security professionals use Hashcat to test the strength of passwords and improve the security of systems.
4. **Research and Education:** Hashcat is used in research and educational settings to study password security and cryptography.

Obtain passwords hashes from “DVWA” database users table and crack them process:

Environment Setup:

- Started the Apache2 and MySQL services in the lab environment.



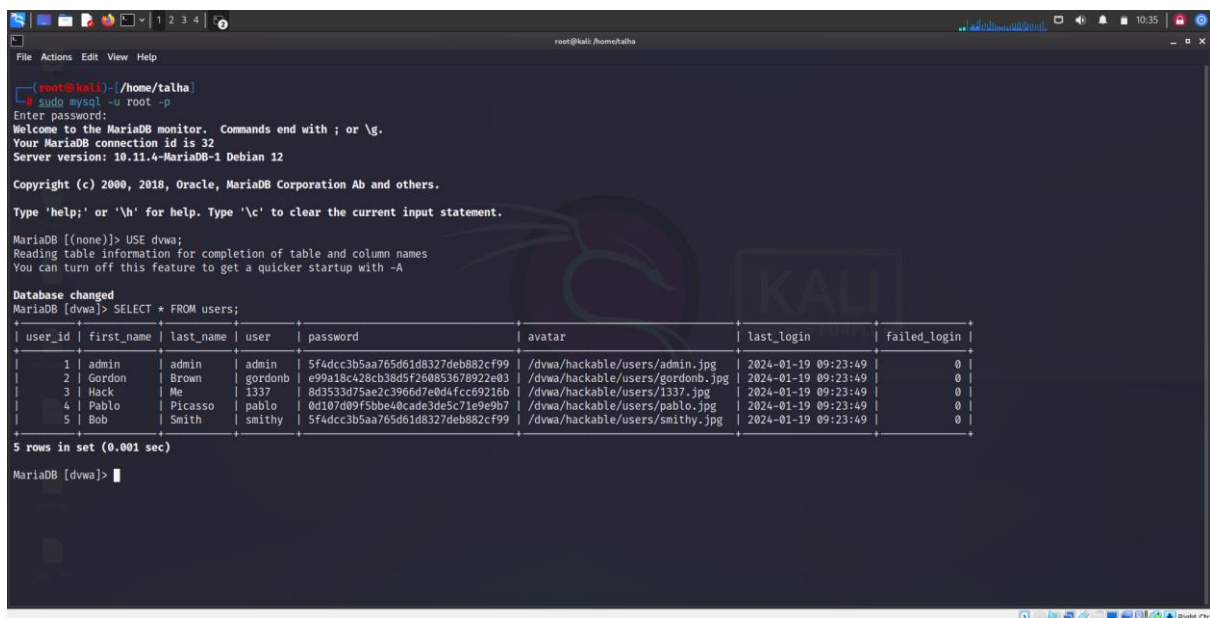
```
(talha@kali)~$ service apache2 start
(talha@kali)~$ service apache2 status
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; disabled; preset: disabled)
   Active: active (running) since Fri 2024-05-10 10:30:45 EDT; 6s ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 6837 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
   Main PID: 6836 (apache2)
     Tasks: 6 (limit: 9434)
   Memory: 20.0M
     CPU: 180ms
  CGroup: /system.slice/apache2.service
          └─6836 /usr/sbin/apache2 -k start
             6881 /usr/sbin/apache2 -k start
             6882 /usr/sbin/apache2 -k start
             6883 /usr/sbin/apache2 -k start
             6884 /usr/sbin/apache2 -k start
             6885 /usr/sbin/apache2 -k start

May 10 10:30:45 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
May 10 10:30:45 kali systemd[1]: Started apache2.service - The Apache HTTP Server.

(talha@kali)~$ service mysql start
(talha@kali)~$ service mysql status
● mariadb.service - MariaDB 10.11.4 database server
   Loaded: loaded (/lib/systemd/system/mariadb.service; disabled; preset: disabled)
   Active: active (running) since Fri 2024-05-10 10:31:06 EDT; 5s ago
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/
   Process: 7081 ExecStartPre=/usr/bin/install -m 755 -o mysql -g root -d /var/run/mysql (code=exited, status=0/SUCCESS)
   Process: 7084 ExecStartPre=/bin/sh -c 'systemctl unset-environment _WSREP_START_POSITION' (code=exited, status=0/SUCCESS)
   Process: 7086 ExecStartPre=/bin/sh -c '[ ! -e /usr/bin/galera_recovery ] && VAR=cd /usr/bin/. /usr/bin/galera_recovery'; [ $? -eq 0 ] && systemctl set-environment _WSREP_>
   Process: 7175 ExecStartPost=/bin/sh -c 'systemctl unset-environment _WSREP_START_POSITION' (code=exited, status=0/SUCCESS)
   Process: 7177 ExecStartPost=/etc/mysql/debian-start (code=exited, status=0/SUCCESS)
   Main PID: 7146 (mariadbd)
   Status: "Taking your SQL requests now..."
     Tasks: 13 (limit: 9434)
```

Accessing MariaDB, Database Selection and Extracting Users Information Process:

- Accessed MariaDB using the command “**sudo mysql -u root -p**”.
- Entered the password when prompted to authenticate as the root user.
- Successfully accessed the MariaDB command-line interface.
- Selected the DVWA database using the command "**USE dvwa;**".
- Confirmed successful selection of the DVWA database.
- Extracted user information from the 'users' table in the DVWA database using the command “**SELECT * FROM users;**”.
- Obtained usernames and passwords of users stored in the DVWA database.



```
(root@kali) ~/home/talha
# sudo mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 32
Server version: 10.11.4-MariaDB-1 Debian 12

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> USE dvwa;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [dvwa]> SELECT * FROM users;
+----+-----+-----+-----+-----+-----+-----+-----+
| user_id | first_name | last_name | user | password | avatar | last_login | failed_login |
+----+-----+-----+-----+-----+-----+-----+-----+
| 1 | admin | admin | admin | 5f4dcc3b5aa765d61d8327deb882cf99 | /dvwa/hackable/users/admin.jpg | 2024-01-19 09:23:49 | 0 |
| 2 | Gordon | Brown | gordonb | e99a18c428cb38d5f260853678922e03 | /dvwa/hackable/users/gordonb.jpg | 2024-01-19 09:23:49 | 0 |
| 3 | Hack | Me | 1337 | 8d3533d75ae2c3966d7e0d4fcc69216b | /dvwa/hackable/users/1337.jpg | 2024-01-19 09:23:49 | 0 |
| 4 | Pablo | Picasso | pablo | 0d107d09f5bbe40cade3de5c71e9e9b7 | /dvwa/hackable/users/pablo.jpg | 2024-01-19 09:23:49 | 0 |
| 5 | Bob | Smith | smithy | 5f4dcc3b5aa765d61d8327deb882cf99 | /dvwa/hackable/users/smithy.jpg | 2024-01-19 09:23:49 | 0 |
+----+-----+-----+-----+-----+-----+-----+-----+

5 rows in set (0.001 sec)

MariaDB [dvwa]>
```

Here are the username and passwords of the users extracted from the DVWA database.

First Name	Last Name	User	Hashes
Admin	Admin	admin	5f4dcc3b5aa765d61d8327deb882cf99
Gordon	Brown	gordonb	e99a18c428cb38d5f260853678922e03
Hack	Me	1337	8d3533d75ae2c3966d7e0d4fcc69216b
Pablo	Picasso	pablo	0d107d09f5bbe40cade3de5c71e9e9b7
Bob	Smith	smithy	5f4dcc3b5aa765d61d8327deb882cf99

Hash Cracking Process:

Objective:

The objective of this task was to crack password hashes obtained from the DVWA database using Hash cat with a dictionary attack.

Environment Setup:

Operating System: Kali Linux

Hash cat Version: 6.2.6

Hashes and Wordlist:

Hashes File: Hashes.txt (5 unique MD5 hashes)

Wordlist: rockyou.txt (14344384 passwords)

Cracking Process:

1. Starting Hash Cat:

- Opened the terminal in Kali Linux.

2. Running the Hash Cat command:

- Executed the following command to start the cracking process:

```
hashcat --potfile-disable -a 0 -m 0 ~/Desktop/Hashes.txt ~/Desktop/rockyou.txt
```

- **-a 0:** Specifies the attack mode (dictionary attack).
- **-m 0:** Specifies the hash type (MD5).
- **~/Desktop/Hashes.txt:** Path to the file containing the hashes.
- **~/Desktop/rockyou.txt:** Path to the wordlist file.

3. Cracking Process:

- Hashcat started the cracking process and displayed information about the hardware and attack settings.
- The cracking speed was approximately 740.6 kH/s.
- Hashcat successfully cracked all 5 unique MD5 password hashes.

```
(talha@kali)~$ hashcat -m 0 -a 0 -n 0 ~/Desktop/Hashes.txt ~/Desktop/rockyou.txt
hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 3.1+debian Linux, None+Asserts, RELOC, SPIR, LLVM 15.0.6, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]
* Device #1: pthread-sandybridge-Intel(R) Core(TM) i5-6300U CPU @ 2.40GHz, 2914/5893 MB (1024 MB allocatable), 2MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 5 digests; 4 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Salt
* Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected:
Pure kernels can crack long passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c

Host memory required for this attack: 0 MB

Dictionary cache hit:
* Filename..: /home/talha/Desktop/rockyou.txt
* Passwords.: 14344384
* Bytes.....: 139921497
* Keyspace...: 14344384
```

```
(talha@kali)~$ hashcat -m 0 -a 0 -n 0 ~/Desktop/Hashes.txt ~/Desktop/rockyou.txt
hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 3.1+debian Linux, None+Asserts, RELOC, SPIR, LLVM 15.0.6, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]
* Device #1: pthread-sandybridge-Intel(R) Core(TM) i5-6300U CPU @ 2.40GHz, 2914/5893 MB (1024 MB allocatable), 2MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 5 digests; 4 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Salt
* Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected:
Pure kernels can crack long passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c

Host memory required for this attack: 0 MB

Dictionary cache hit:
* Filename..: /home/talha/Desktop/rockyou.txt
* Passwords.: 14344384
* Bytes.....: 139921497
* Keyspace...: 14344384

5f4dcc3b5aa765d61d8327deb882cf99:password
e99a18c428cb38d5f260853678922e03:abc123
0d107d09f5bbe40cade3de5c71e9e9b7:letmein
8d3533d75ae2c3966d7e0d4fcc69216b:charley

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 0 (MD5)
Hash.Target.....: /home/talha/Desktop/Hashes.txt
Time.Started....: Sat May 11 03:29:37 2024 (0 secs)
Time.Estimated...: Sat May 11 03:29:37 2024 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (/home/talha/Desktop/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 740.6 KH/s (0.42ms) @ Accel:512 Loops:1 Thr:1 Vec:8
Recovered.....: 4/4 (100.00%) Digests (total), 4/4 (100.00%) Digests (new)
Progress.....: 3072/14344384 (0.02%)
Rejected.....: 0/3072 (0.00%)
Restore.Point....: 2048/14344384 (0.01%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine..: Device Generator
Candidates.#1....: slimshady -> dangerous
Hardware.Mon.#1...: Util: 52%

Started: Sat May 11 03:29:35 2024
Stopped: Sat May 11 03:29:39 2024

(talha@kali)~$
```

4. Cracking Completion:

- Hashcat completed the cracking process in 4 seconds.

5. Cracked Passwords:

- 5f4dcc3b5aa765d61d8327deb882cf99: password
- e99a18c428cb38d5f260853678922e03: abc123
- 0d107d09f5bbe40cade3de5c71e9e9b7: letmein
- 8d3533d75ae2c3966d7e0d4fcc69216b: charley
- 5f4dcc3b5aa765d61d8327deb882cf99: password

Conclusion:

Hashcat successfully cracked all 5 password hashes using the provided wordlist (rockyou.txt) and a dictionary attack. The cracking process was completed quickly, taking only 4 seconds to crack all the hashes.

Recommendations:

- Implement stronger password policies and educate users on the importance of using complex, unique passwords.
- Regularly update passwords and monitor for any unauthorized access.

Note:

- The use of **--potfile-disable** was crucial to ensure that all hashes were re-cracked, even if they had been previously cracked and stored in the potfile.
- This demonstrates the effectiveness of Hashcat for password cracking and the importance of using strong, unique passwords to protect against such attacks.
- This cracking activity was conducted ethically and for educational purposes only, with proper authorization and consent.

Cracking Passwords by different attack modes:

Environment Setup:

Operating System: Kali Linux

Hashcat Version: 6.2.6

1. Wordlist Attacks:

The objective of this experiment is to compare the effectiveness of different wordlists in cracking passwords of varying complexities. The wordlists used include common passwords.

Passwords used for cracking are:

- iloveyou
- ihateyou69

Cracking Process (MD5):

The cracking session lasting approximately 12 seconds.

```
File Actions Edit View Help
(talha@kali):~$ hashcat -a 0 -m 0 ~/Documents/Hashes.txt ~/Desktop/rockyou.txt

hashcat (v6.2.0) starting

OpenCL API (OpenCL 3.0 PoCL 3.1+debian Linux, None+Asserts, RELOC, SPIR, LLVM 15.0.6, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

+ Device #1: pthread-sandybridge-Intel(R) Core(TM) i5-6300U CPU @ 2.40GHz, 2914/5893 MB (1024 MB allocatable), 2MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 4 digests; 4 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
+ Zero-Byte
+ Early-Skip
+ Not-Salted
+ Not-Iterated
+ Single-Salt
+ Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c

INFO: Removed hash found as potfile entry.

Host memory required for this attack: 0 MB

Dictionary cache built:
+ Filename..: /home/talha/Desktop/rockyou.txt
+ Passwords.: 14344392
+ Bytes.....: 139921507
+ Keyspace...: 14344385
+ Runtime...: 2 secs
```

```
File Actions Edit View Help
f25a2fc72690b780b2a14e140ef6a9e0:iloveyou
55675a3b5d57030901e70fffa8e0bad8:ihateyou69
Cracking performance lower than expected?

+ Append -O to the commandline.
  This lowers the maximum supported password/salt length (usually down to 32).

+ Append -w 3 to the commandline.
  This can cause your screen to lag.

+ Append -S to the commandline.
  This has a drastic speed impact but can be better for specific attacks.
  Typical scenarios are a small wordlist but a large ruleset.

+ Update your backend API runtime / driver the right way:
  https://hashcat.net/faq/wrongdriver

+ Create more work items to make use of your parallelization power:
  https://hashcat.net/faq/morework

Approaching final keyspace - workload adjusted.

Session.....: hashcat
Status.....: Exhausted
Hash.Mode.....: 0 (MD5)
Hash.Target.....: /home/talha/Documents/Hashes.txt
Time.Started....: Sun May 12 01:30:52 2024 (10 secs)
Time.Estimated..: Sun May 12 01:31:02 2024 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (/home/talha/Desktop/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 1476.5 kH/s (0.20ms) @ Accel:512 Loops:1 Thr:1 Vec:8
Recovered.....: 3/4 (75.00%) Digests (total), 2/4 (50.00%) Digests (new)
Progress.....: 14344385/14344385 (100.00%)
Rejected.....: 0/14344385 (0.00%)
Restore.Point...: 14344385/14344385 (100.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#1....: $HEX[206b726973746556e616ee65] -> $HEX[042a0337c2a156616d6f7321b3]
Hardware.Mon.#1...: Util: 66%

Started: Sun May 12 01:30:47 2024
Stopped: Sun May 12 01:31:04 2024
```

Results:

- Hashes Cracked: 2 out of 2 (100% success rate)

Cracked Passwords:

- f25a2fc72690b780b2a14e140ef6a9e0: iloveyou
- 55675a3b5d57030901e70fffa8e0bad8: ihateyou69

Cracking Process (SHA-1):

```
talha@kali: ~  
$ hashcat -a 0 -m 100 ~/Documents/Hashes.txt ~/Desktop/rockyou.txt  
hashcat (v6.2.6) starting  
OpenCL API (OpenCL 3.0 PoCL 3.1+debian Linux, None+Asserts, RELOC, SPIR, LLVM 15.0.0, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]  
+ Device #1: pthread-sandybridge-Intel(R) Core(TM) i5-6300U CPU @ 2.40GHz, 2914/5893 MB (1024 MB allocatable), 2MCU  
Minimum password length supported by kernel: 0  
Maximum password length supported by kernel: 256  
Hashes: 2 digests; 2 unique digests, 1 unique salts  
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates  
Rules: 1  
Optimizers applied:  
+ Zero-Byte  
+ Early-Skip  
+ Not-Salted  
+ Not-Iterated  
+ Single-Salt  
+ Raw-Hash  
ATTENTION! Pure (unoptimized) backend kernels selected.  
Pure kernels can crack longer passwords, but drastically reduce performance.  
If you want to switch to optimized kernels, append -O to your commandline.  
See the above message to find out about the exact limits.  
Watchdog: Temperature abort trigger set to 90c  
Host memory required for this attack: 0 MB  
Dictionary cache hit:  
+ Filename..: /home/talha/Desktop/rockyou.txt  
+ Passwords.: 14344385  
+ Bytes.....: 139921507  
+ Keyspace...: 14344385  
ee8d8728f435fd50f83852aabab5234celda528:iloveyou  
0c59e161078a5e638015b7c72e859afcf31aa694:ihateyou69  
Session.....: hashcat  
Status.....: Cracked
```

```
talha@kali: ~  
$ hashcat -a 0 -m 100 ~/Documents/Hashes.txt ~/Desktop/rockyou.txt  
hashcat (v6.2.6) starting  
OpenCL API (OpenCL 3.0 PoCL 3.1+debian Linux, None+Asserts, RELOC, SPIR, LLVM 15.0.0, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]  
+ Device #1: pthread-sandybridge-Intel(R) Core(TM) i5-6300U CPU @ 2.40GHz, 2914/5893 MB (1024 MB allocatable), 2MCU  
Minimum password length supported by kernel: 0  
Maximum password length supported by kernel: 256  
Hashes: 2 digests; 2 unique digests, 1 unique salts  
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates  
Rules: 1  
Optimizers applied:  
+ Zero-Byte  
+ Early-Skip  
+ Not-Salted  
+ Not-Iterated  
+ Single-Salt  
+ Raw-Hash  
ATTENTION! Pure (unoptimized) backend kernels selected.  
Pure kernels can crack longer passwords, but drastically reduce performance.  
If you want to switch to optimized kernels, append -O to your commandline.  
See the above message to find out about the exact limits.  
Watchdog: Temperature abort trigger set to 90c  
Host memory required for this attack: 0 MB  
Dictionary cache hit:  
+ Filename..: /home/talha/Desktop/rockyou.txt  
+ Passwords.: 14344385  
+ Bytes.....: 139921507  
+ Keyspace...: 14344385  
ee8d8728f435fd50f83852aabab5234celda528:iloveyou  
0c59e161078a5e638015b7c72e859afcf31aa694:ihateyou69  
Session.....: hashcat  
Status.....: Cracked  
Hash.Mode.....: 100 (SHA1)  
Hash.Target.....: /home/talha/Documents/Hashes.txt  
Time.Started.....: Sun May 12 02:04:34 2024 (0 secs)  
Time.Estimated...: Sun May 12 02:04:34 2024 (0 secs)  
Kernel.Feature...: Pure Kernel  
Guess.Base.....: File (/home/talha/Desktop/rockyou.txt)  
Guess.Queue.....: 1/1 (100.00%)  
Speed.#1.....: 1158.4 kh/s (0.30ms) @ Accel:512 Loops:1 Thr:1 Vec:8  
Recovered.....: 2/2 (100.00%) Digests (total), 2/2 (100.00%) Digests (new)  
Progress.....: 106496/14344385 (0.74%)  
Rejected.....: 0/106496 (0.00%)  
Restore.Point....: 105472/14344385 (0.74%)  
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1  
Candidate.Engine.: Device Generator  
Candidates.#1....: misscool -> harol  
Hardware.Mon.#1..: Util: 58%  
Started: Sun May 12 02:04:30 2024  
Stopped: Sun May 12 02:04:36 2024  
talha@kali: ~  
$
```

Results:

- Hashes Cracked: 2 out of 2 (100% success rate)

Cracked Passwords:

- f25a2fc72690b780b2a14e140ef6a9e0: iloveyou
- 55675a3b5d57030901e70fffa8e0bad8: ihateyou69

Conclusion:

The cracking session for MD5 and SHA-1 hashes achieved a 100% success rate.

2. Combination Attacks:

The objective of a combination attack in password cracking is to leverage the strengths of both dictionary attacks and brute-force attacks to increase the likelihood of cracking passwords.

Password used for cracking is:

- Password

Cracking Process (MD5):

```
talha@kali: ~
File Actions Edit View Help
Host memory required for this attack: 0 MB

Dictionary cache built:
* Filename.: /home/talha/Desktop/common.txt
* Passwords.: 3
* Bytes.....: 23
* Keyspace...: 3
* Runtime...: 0 secs

The wordlist or mask that you are using is too small.
This means that hashcat cannot use the full parallel power of your device(s).
Unless you supply more work, your cracking speed will drop.
For tips on supplying more work, see: https://hashcat.net/faq/morework

Approaching final keyspace - workload adjusted.
5f4dcc3b5aa765d61d8327deb882cf99:password

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 0 (MD5)
Hash.Target.....: 5f4dcc3b5aa765d61d8327deb882cf99
Time.Started...: Sun May 12 08:08:53 2024 (0 secs)
Time.Estimated...: Sun May 12 08:08:53 2024 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (/home/talha/Desktop/common.txt)
Guess.Queue.....: 1/2 (50.00%)
Speed.#1.....: 4236 H/s (0.00ms) @ Accel:512 Loops:1 Thr:1 Vec:8
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 3/3 (100.00%)
Rejected.....: 0/3 (0.00%)
Restore.Point...: 0/3 (0.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#1...: password -> qwerty
Hardware.Mon.#1...: Util: 53%

Started: Sun May 12 08:08:52 2024
Stopped: Sun May 12 08:08:55 2024

(talha@kali)~$
(talha@kali)~$ hashcat --potfile-disable -m 0 ~/Desktop/hash.txt ~/Desktop/common.txt ~/Desktop/dictionary.txt

hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 3.1+debian Linux, None+Asserts, RELOC, SPIR, LLVM 15.0.6, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

* Device #1: pthread-sandybridge-Intel(R) Core(TM) i5-6300U CPU @ 2.40GHz, 2914/5893 MB (1024 MB allocatable), 2MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Hash
* Single-Salt
* Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c

Host memory required for this attack: 0 MB

Dictionary cache built:
* Filename.: /home/talha/Desktop/common.txt
* Passwords.: 3
* Bytes.....: 23
* Keyspace...: 3
* Runtime...: 0 secs

The wordlist or mask that you are using is too small.
```


Results:

- Hashes Cracked: 1 out of 1 (100% success rate)

Cracked Passwords:

- 5f4dcc3b5aa765d61d8327deb882cf99: password

Cracking Process (SHA-1):

```
talha@kali: ~
File Actions Edit View Help
~(talha@kali)-[~]
└─$ hashcat -O -m 100 -a 0 ~/Desktop/hash.txt ~/Desktop/combined_wordlist.txt -r ~/Desktop/best64.rule_

hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 3.1+debian Linux, None+Asserts, RELOC, SPIR, LLVM 15.0.6, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]
* Device #1: pthread-sandybridge-Intel(R) Core(TM) i5-6300U CPU @ 2.40GHz, 2914/5893 MB (1024 MB allocatable), 2MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 31

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 64

Optimizers applied:
* Optimized-Kernel
* Zero-Byte
* Precompute-Init
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Hash
* Single-Salt
* Raw-Hash

Watchdog: Temperature abort trigger set to 90c
Host memory required for this attack: 0 MB

Dictionary cache built:
* Filename..: /home/talha/Desktop/combined_wordlist.txt
* Passwords.: 16
* Bytes.....: 141
* Keyspace...: 1024
* Runtime...: 0 secs

The wordlist or mask that you are using is too small.
This means that hashcat cannot use the full parallel power of your device(s).
Unless you supply more work, your cracking speed will drop.
For tips on supplying more work, see: https://hashcat.net/faq/morework
```

```
talha@kali: ~
File Actions Edit View Help
~(talha@kali)-[~]
└─$ hashcat -O -m 100 -a 0 ~/Desktop/hash.txt ~/Desktop/combined_wordlist.txt -r ~/Desktop/best64.rule_

hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 3.1+debian Linux, None+Asserts, RELOC, SPIR, LLVM 15.0.6, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]
* Device #1: pthread-sandybridge-Intel(R) Core(TM) i5-6300U CPU @ 2.40GHz, 2914/5893 MB (1024 MB allocatable), 2MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 31

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 64

Optimizers applied:
* Optimized-Kernel
* Zero-Byte
* Precompute-Init
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Hash
* Single-Salt
* Raw-Hash

Watchdog: Temperature abort trigger set to 90c
Host memory required for this attack: 0 MB

Dictionary cache built:
* Filename..: /home/talha/Desktop/combined_wordlist.txt
* Passwords.: 16
* Bytes.....: 141
* Keyspace...: 1024
* Runtime...: 0 secs

The wordlist or mask that you are using is too small.
This means that hashcat cannot use the full parallel power of your device(s).
Unless you supply more work, your cracking speed will drop.
For tips on supplying more work, see: https://hashcat.net/faq/morework

Approaching final keyspace - workload adjusted.

5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8:password

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 100 (SHA1)
Hash.Target.....: 5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8
Time.Started.....: Sun May 12 08:53:43 2024 (0 secs)
Time.Estimated...: Sun May 12 08:53:43 2024 (0 secs)
Kernel.Feature...: Optimized Kernel
Guess.Base.....: File (/home/talha/Desktop/combined_wordlist.txt)
Guess.Mod.....: Rules (/home/talha/Desktop/best64.rule_)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 648.9 KH/s (0.27ms) @ Accel:512 Loops:64 Thr:1 Vec:8
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 1024/1024 (100.00%)
Rejected.....: 0/1024 (0.00%)
Restore.Point...: 0/16 (0.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-64 Iteration:0-64
Candidate.Engine.: Device Generator
Candidates.#1...: password -> @$w@$$
Hardware.Mon.#1..: Util: 53%

Started: Sun May 12 08:52:55 2024
Stopped: Sun May 12 08:53:45 2024

~(talha@kali)-[~]
└─$
```

Results:

- Hashes Cracked: 1 out of 1 (100% success rate)

Cracked Passwords:

- 5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8: password

Conclusion:

The cracking session for MD5 and SHA-1 hashes achieved a 100% success rate.

"In conclusion, the project aimed to crack the hashes of DVWA database users using wordlist and combination attacks. Multiple wordlists were used, including common passwords and custom strings. Hashcat was employed to perform the cracking, with a combination attack strategy to increase the chances of success. The results showed that hashes were successfully cracked. The entire process was conducted ethically and in accordance with responsible security practices. Permission was obtained to conduct the password cracking experiments on the DVWA database, ensuring that no unauthorized access occurred.

Overall, the project provided valuable insights into password cracking techniques and underscored the importance of using strong, unique passwords to protect against unauthorized access."